

# Failure as a Process: An Anatomy of CLI Coding Agent Trajectories

[https://arxiv.org/abs/2607.09510?utm\\_source=substack&utm\\_medium=email](https://arxiv.org/abs/2607.09510?utm_source=substack&utm_medium=email)

## 1. Problem

- LLM 기반 코딩 에이전트는 터미널 환경에서 실제 소프트웨어 개발 작업을 수행하지만 실패 원인과 발생 과정에 대한 이해가 부족함
- 기존 연구는 대부분 최종 성공/실패 결과만 분석하고 실패가 언제 시작되고 어떻게 누적되는지는 다루지 않음
- 중요한 문제는 “왜 실패했는가”보다 “실패가 어떤 과정으로 발생하고 복구 불가능해지는가”를 이해하는 것

## 2. Method

- CLI 코딩 에이전트의 실행 과정을 failure trajectory (실패 궤적) 관점에서 분석하는 프레임워크 제안
- 실패를 세 단계로 구분:
  - Failure onset: 실패가 처음 발생하는 시점
  - Failure evolution: 잘못된 행동이 어떻게 누적되는지
  - Failure recovery: 에이전트가 오류를 감지하고 복구하는 과정
- 실험 데이터:
  - 7개 최신 LLM 기반 에이전트 분석
  - 3개 코딩 에이전트 프레임워크 사용
  - Terminal-Bench 기반 실행 trajectory 수집
- 총 3,843개 실행 trajectory 분석
- 1,794개 유효 trajectory 선별

- 63,000개 이상의 실행 step 수동 annotation 수행

### 3. Experiments & Results

- 분석 대상:
  - 에이전트가 터미널 환경에서 수행하는 실제 coding task
  - command 실행, reasoning, error handling 과정 분석
- 주요 결과:
  - 코딩 에이전트 실패 대부분은 epistemic error (상황 이해 부족, 잘못된 판단)에서 발생
  - 단순 command 오류보다 문제 상황에 대한 잘못된 추론이 주요 원인
  - 실패는 대부분 초기 몇 단계 안에 시작
  - 초반 판단 오류가 이후 trajectory 전체에 영향을 줌
  - 많은 실패는 초기에 감지되지 않고 진행
  - 에이전트가 잘못된 방향으로 계속 실행하다가 복구 불가능 상태에 도달
  - 최종 결과 평가보다:
    - early validation
    - intermediate monitoring
    - trajectory-level feedback
 이 중요하다는 것 제시

### 4. Insight

- 핵심 기여
  - LLM coding agent의 실패를 결과가 아닌 과정(process)으로 분석한 대규모 empirical study
  - 안정적인 coding agent 개발을 위해서는 마지막 결과 평가보다 실행 중 오류 탐지가 중요
- 한계점
  - Terminal-Bench 기반 분석으로 실제 개발 환경 전체를 대표하지 못할 가능성
  - 실패 원인 annotation이 사람 판단에 의존